Newton

Inovação é a nossa tradição.

Newton
Inovação é a nossa tradição.

Estruturas de Dados

Prof. Fabrício Rodrigues Inácio
fabricio.inacio@professores.newtonpaiva.edu.br

Vetor – Vantagens e Desvantagens

Vantagens de se utilizar vetores:

- Fácil implementação comparada a listas encadeadas.
- Redução da quantidade de memória utilizada, pois não requer armazenar a localização do próximo nodo.
- A sua dimensão está disponível sem a necessidade de percorrer todos os nodos.

Desvantagens na utilização de vetores:

- Dificuldade na ordenação dos seus elementos.
- O aumento do seu tamanho poderá requerer o deslocamento do vetor em memória, caso não haja espaço de forma linear.
- A impossibilidade de adicionar um elemento no meio do vetor sem realizar o deslocamento de todos os demais nodos, logo, consumindo alto processamento.

Estruturas de Dados Dinâmicas

As estruturas dinâmicas são utilizadas para relacionar itens e nodos que precisam ser manipulados em tempo de execução com dimensão indefinida. A partir dessa premissa, podemos dizer que durante o tempo de execução será possível adicionar ou remover itens de uma estrutura dinâmica de dados de acordo com a necessidade do usuário.

Listas Encadeadas

Na computação, uma lista pode ser definida como um conjunto de elementos do mesmo tipo, agrupados e identificados por um identificador único e separados entre si em “caixas”, chamados de nodos, que ocupam um endereço específico na memória.

O relacionamento entre os nodos é definido por sua posição em relação aos demais nodos, assim como pessoas em uma fila, porém indexados na memória do computador.

Listas Encadeadas

Toda lista apresenta um nodo inicial, o primeiro elemento da lista. A partir deste seguirá uma sequência de nodos conforme uma ordem predefinida pelo programador, assim como uma fila de banco.

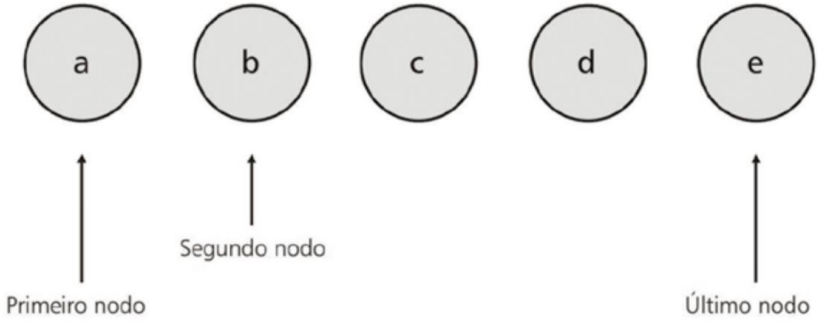


Figura 1. Lista composta por cinco elementos, nodos, sendo cada um identificado por uma letra do alfabeto.

Fonte: Edelweiss e Galante (2009, p. 50).

Estruturas de Listas Encadeadas

O elemento básico de uma lista simplesmente encadeada é o nodo, cuja definição e função é importante compreender antes de tentar entender uma lista inteira.

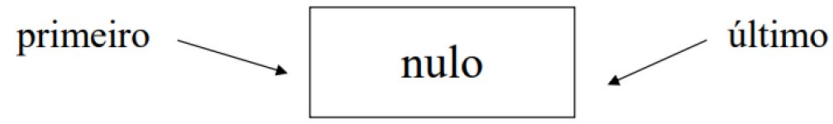
Em Java, um nodo pode ser representado por uma classe com dois atributos, um dado e uma referência para o próximo elemento da lista.

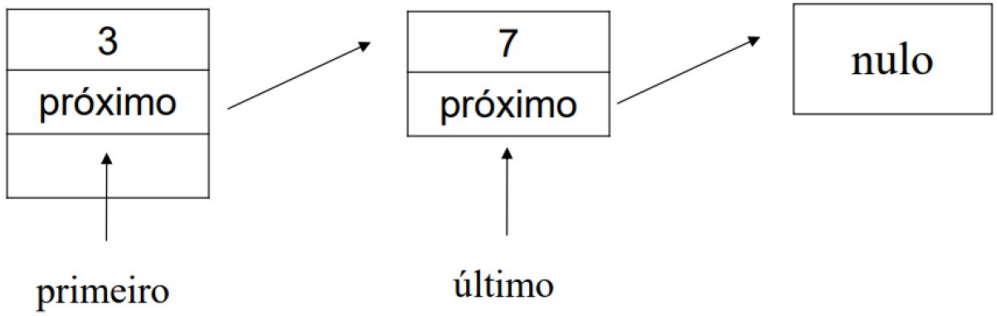
De acordo com Knuth (1997), o armazenamento de informações em nodos e referências é mais flexível que a alocação de memória sequencial realizada por arrays, facilitando a inserção e a exclusão de informações, embora, como ponto negativo, tenha uma maior necessidade de memória.

Operações com listas encadeadas

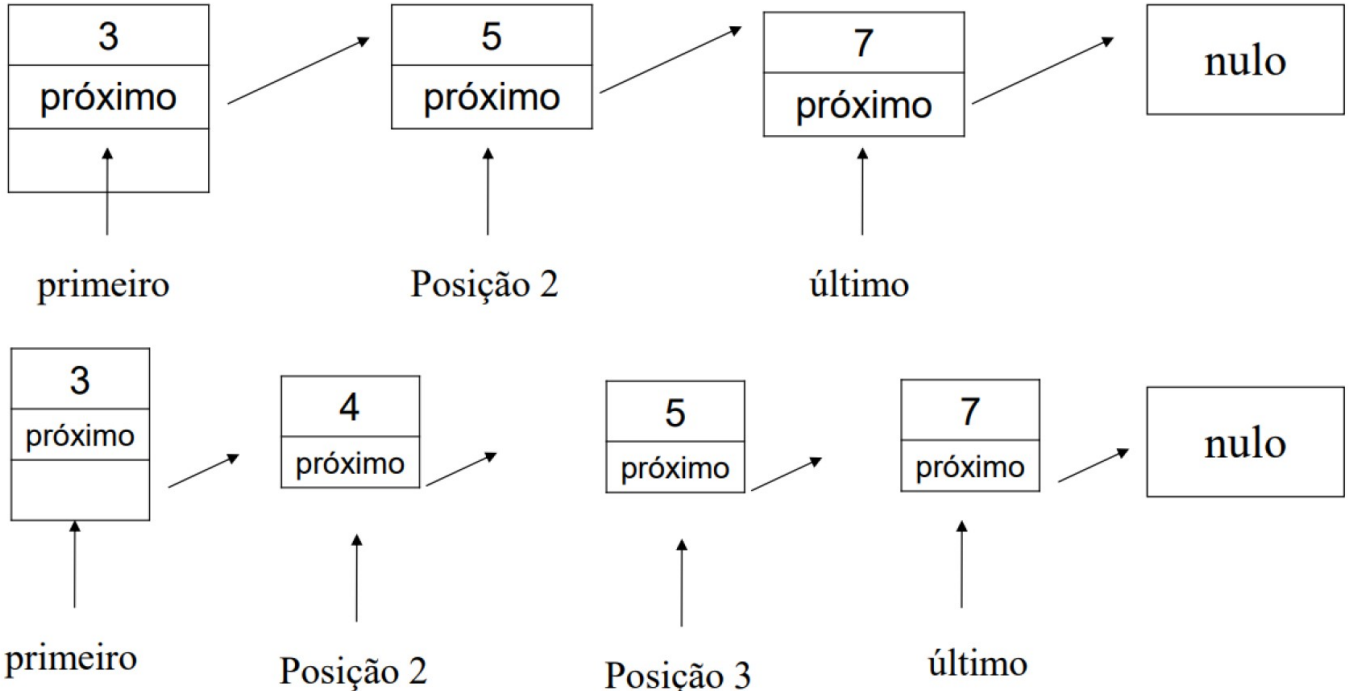
- Inserir no início – insere um elemento no início da lista.
- Inserir no meio – insere um elemento no meio da lista.
- Inserir no final – insere um elemento no final da lista.
- Lista vazia – verifica se a lista está vazia.
- Elemento do início – mostra o elemento que está no início da lista.
- Elemento do final – mostra o elemento que está no final da lista.
- Remover – elimina um elemento que está na lista ligada.
- Contar nós – verifica quantos elementos existem na lista.
- Mostrar lista – exibe todos os elementos que a lista possui.
- Buscar – verifica se determinado elemento pertence à lista.
- Destruir – elimina todos os elementos da lista ligada.

Simulação de listas encadeadas

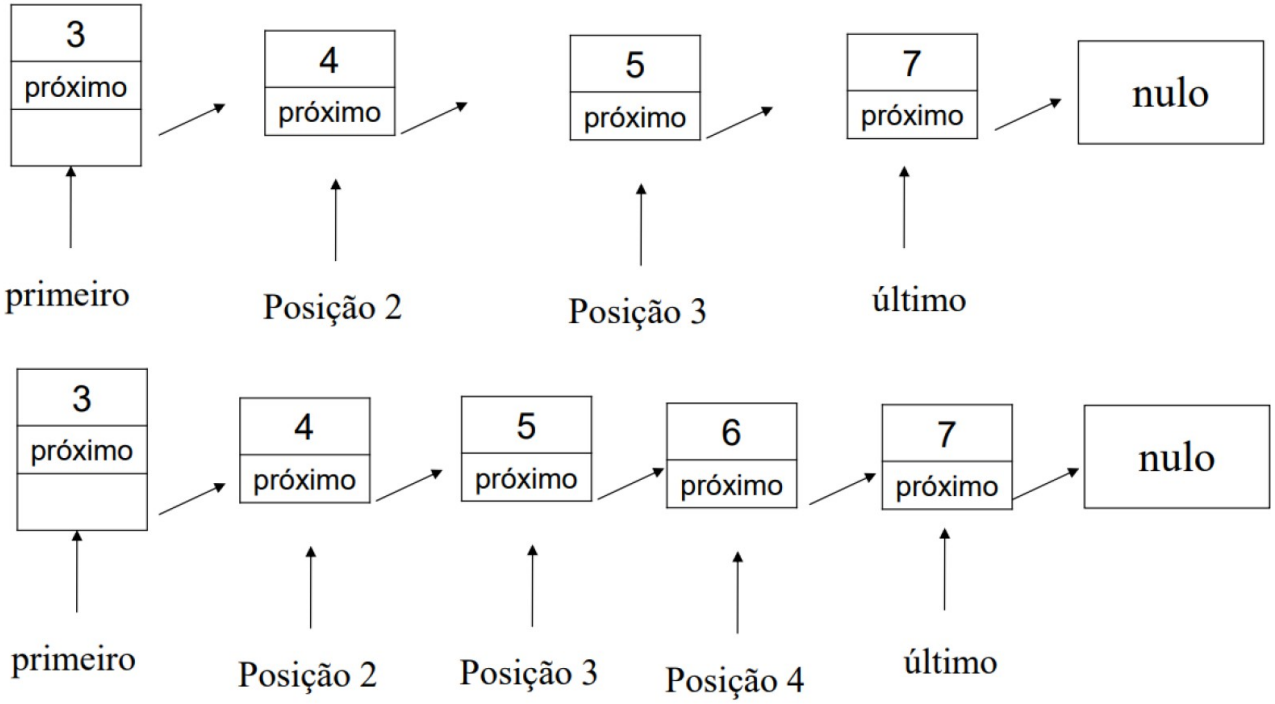




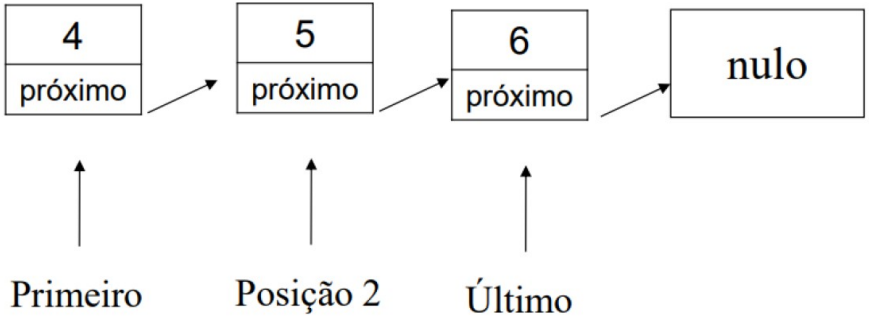
Simulação de listas encadeadas

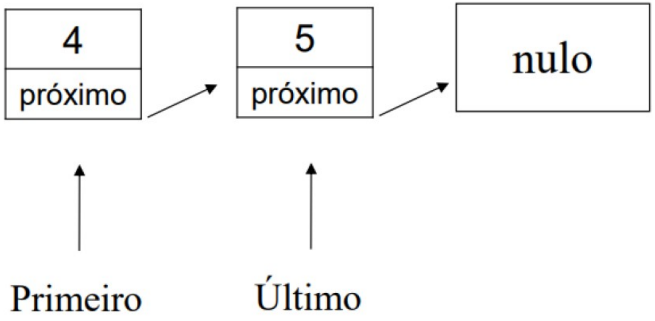


Simulação de listas encadeadas



Simulação de listas encadeadas





Implementação de listas encadeadas

```java
public class Nodo {
   private float valor;
   private Nodo proximo;

   public float getValor (){
      return valor;
   }

   public void setValor(float valor){
      this.valor = valor;
   }

   public Nodo getProximo(){
      return proximo;
   }

   public void proximo (Nodo proximo){
      this.proximo = proximo;
   }
}
```

```java
public class Principal {
   public static void main(String[] args ){
      Nodo nodo7 = new Nodo();
      nodo7.setValor(7);
      Nodo nodo8 = new Nodo();
      nodo8.setValor(8);
      nodo7.setProximo(nodo8);
      System.out.println(nodo7.getValor());
      System.out.println(nodo7.proximo());
   }
}
```